

Tuition Class and Student Management System

Mr. (Eng.) D.S. De Silva

Department of Electrical and Information Engineering
Faculty of Engineering, University of Ruhuna
Galle, Sri Lanka
saman@eie.ruh.ac.lk

D.P.S.T. Kumari

Department of Electrical and Information Engineering
Faculty of Engineering, University of Ruhuna
Galle, Sri Lanka
kumari_dpst_e22@engug.ruh.ac.lk

W.M.M.T. Wijerathna

Department of Electrical and Information Engineering
Faculty of Engineering, University of Ruhuna
Galle, Sri Lanka
wijerathna_wmmt_e22@engug.ruh.ac.lk

R.M.B.T.M. Rathnayake

Department of Electrical and Information Engineering
Faculty of Engineering, University of Ruhuna
Galle, Sri Lanka
rathnayake_rmbtm_e22@engug.ruh.ac.lk

R.D. Randula

Department of Electrical and Information Engineering
Faculty of Engineering, University of Ruhuna
Galle, Sri Lanka
randula_rd_e22@engug.ruh.ac.lk

Abstract—Traditional tuition management systems lack the capability to handle both online and physical classes simultaneously. This creates operational challenges for educational institutions. We present a comprehensive hybrid tuition class and student management system. The system addresses these challenges through a microservices architecture. It supports unified class delivery with verified tracking. The platform integrates React frontend with PHP microservices backend. It uses Docker for deployment and MySQL databases for data storage. The system ensures security through JSON Web Tokens (JWT), Role-Based Access Control (RBAC), and audit logs. Key features include automated attendance tracking using Zoom webhooks and barcode scanning. The system provides WhatsApp-based communication for OTP verification, receipts, and reminders. The platform delivers REST APIs for external integrations. The system runs on AWS infrastructure with Jenkins CI/CD pipeline. It enables zero-downtime deployments. The platform is currently deployed at app.tcmslk.me. It serves real tuition classes successfully. Results show improved administrative efficiency and enhanced student engagement. The system provides financial transparency and automated workflows. This research demonstrates that microservices architecture can effectively support hybrid educational environments. It combines scalability, security, and intelligent automation.

Index Terms—Tuition Management System, Microservices Architecture, Hybrid Learning, Docker Containerization, REST API, JWT Authentication, RBAC, Cloud Deployment, Payment Integration

I. INTRODUCTION

A. Background

Educational institutions worldwide are adopting hybrid learning models. These models combine traditional classroom teaching with online delivery. Tuition centers in Sri Lanka face unique challenges. They must manage both physical and online classes simultaneously. Traditional management systems cannot handle this complexity effectively.

The COVID-19 pandemic accelerated the need for hybrid solutions. Many institutions struggled with fragmented tools. They used separate systems for attendance, payments, and communication. This created inefficiencies and data inconsistencies. Students and teachers faced difficulties accessing information. Administrative staff spent excessive time on manual tasks.

B. Problem Statement

Current tuition management systems have several limitations:

- **Limited hybrid support:** Most systems support either online or physical classes, not both simultaneously.
- **Fragmented attendance tracking:** Separate mechanisms exist for online and offline attendance without unified reporting.
- **Manual administrative burden:** Staff waste time on repetitive tasks like scheduling, reminders, and conflict detection.
- **Poor financial transparency:** Students and parents cannot track payments in real-time.
- **Weak communication:** No integrated messaging system for instant updates and notifications.
- **Security concerns:** Basic authentication without role-based permissions or audit trails.
- **Scalability issues:** Monolithic architectures cannot handle growing user bases efficiently.

C. Motivation

Industry partner Mrs. Nilanka Dilhani operates a tuition center. He faced these exact problems daily. Manual attendance marking consumed significant time. Payment tracking created

disputes. Scheduling conflicts occurred frequently. Students missed classes due to poor communication.

This real-world requirement motivated our research. We aimed to build a production-ready solution. The system needed to be scalable, secure, and user-friendly. It had to reduce administrative overhead significantly. Most importantly, it needed to work reliably in production.

D. Research Objectives

Our research aims to achieve the following objectives:

- 1) Design an easy-to-use platform for student enrollment, attendance tracking, and class scheduling.
- 2) Implement unified attendance tracking for both online and physical classes using modern technologies.
- 3) Develop a financial management module with real-time payment tracking and transparent reporting.
- 4) Create automated report generation for student details, attendance records, and financial transactions.
- 5) Build an integrated communication system using WhatsApp for broadcasting and notifications.
- 6) Automate administrative tasks including scheduling, reminders, and conflict detection.
- 7) Apply enterprise-grade security using JWT authentication, RBAC, and comprehensive audit logging.
- 8) Deploy the system using modern DevOps practices for scalability and zero-downtime updates.

E. Paper Organization

The remainder of this paper is organized as follows. Section II describes the system architecture and design principles. Section III details the implementation of each microservice. Section IV presents the results and performance evaluation. Section V discusses the findings and their implications. Section VI concludes the paper and outlines future work.

II. SYSTEM ARCHITECTURE AND DESIGN

A. Architecture Overview

We designed the system using microservices architecture. This approach provides several advantages. Each service handles a specific business function. Services communicate through REST APIs. They can be developed, deployed, and scaled independently.

The architecture consists of five main layers:

- 1) **Client Layer:** Web browsers access the React-based frontend.
- 2) **API Gateway Layer:** Nginx reverse proxy routes requests to appropriate services.
- 3) **Microservices Layer:** Eight independent services handle business logic.
- 4) **Data Layer:** Each service has its own MySQL database instance.
- 5) **Integration Layer:** External APIs for payments, messaging, and video conferencing.

Fig. 1 illustrates the system context showing all actors and external integrations. The context diagram demonstrates how different user roles interact with the system. Students, teachers,

administrators, cashiers, and staff members access the platform through web interfaces. External services include WhatsApp for communication, Zoom for online classes, and PayHere for payment processing. The hardware layer represents barcode scanners used for physical attendance.

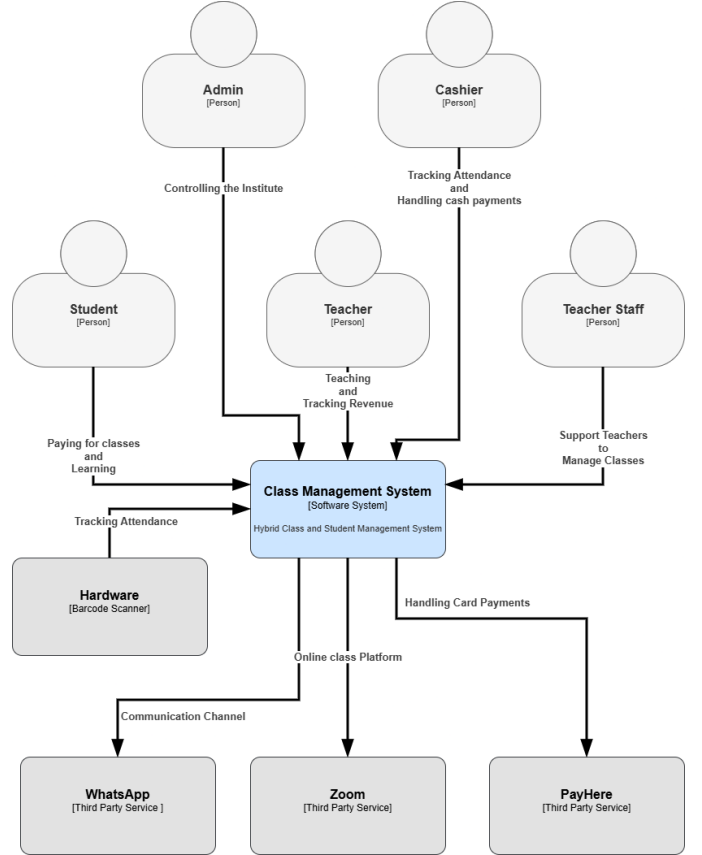


Fig. 1. System Context Diagram showing users, external services, and hardware integration.

Fig. 2 presents the complete system architecture. The diagram shows the client layer with React frontend on port 3000. Cloudflare CDN provides content delivery and DDoS protection. Nginx reverse proxy with SSL handles routing to backend services. The backend layer contains eight PHP microservices with dedicated MySQL databases. External integrations include PayHere Gateway, Zoom Webhooks, and WhatsApp API. The deployment layer shows Docker containers, Jenkins CI/CD, and AWS EC2 production environment.

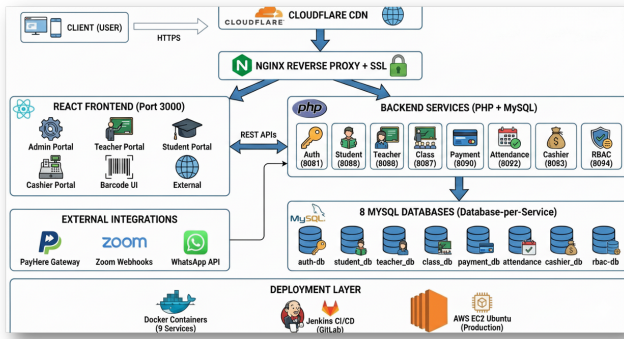


Fig. 2. Complete system architecture showing frontend, backend services, databases, and deployment infrastructure.

B. Microservices Components

1) *Authentication Service*: The auth service handles all authentication operations. It manages user registration and login. The service issues JWT tokens for secure sessions. It implements refresh token rotation for enhanced security. OTP verification uses WhatsApp integration. The service maintains separate tables for users, students, teachers, and administrators.

2) *Student Service*: This service manages student-related operations. It stores student profiles and academic records in the student database. The service integrates with the auth service for barcode management. The BarcodeController in the auth service handles barcode generation and storage. Students receive unique barcode identifiers for physical attendance tracking. The service provides REST APIs for student enrollment, profile updates, and data retrieval.

3) *Teacher Service*: The teacher service manages teacher profiles and operations. It handles hall booking requests with conflict detection. Teachers can upload teaching materials and recordings. The service manages exam paper creation and distribution. It integrates with the class service for schedule management.

4) *Class Service*: This is the core service for class management. It handles class creation, scheduling, and updates through the ClassController. The EnrollmentController manages student enrollments and capacity limits. The LatePayController implements late payment permissions allowing students to attend classes pending payment. The EntryPermitController generates entry permits for students with valid enrollment. The service implements conflict detection to prevent double bookings. The service exposes REST APIs at endpoints like /class/*, /enrollment/*, /late_pay/*, and /entry_permit/.*.

5) *Payment Service*: The payment service integrates with PayHere gateway. It processes online payments securely. The service tracks all financial transactions. It generates payment receipts automatically. Students can view their payment history in real-time. The service supports installment payments and discounts.

6) *Cashier Service*: This service handles cash payments at the physical location. It manages cashier sessions and day-end

reports. The service tracks cash collections and reconciliations. It integrates with the payment service for unified financial reporting. Audit logs track all cash transactions.

7) *Attendance Service*: The attendance service supports both online and offline tracking. For online classes, it integrates with Zoom webhooks. Zoom automatically marks attendance when students join. For physical classes, it uses barcode scanning. Students scan their ID cards at the entrance. The service generates comprehensive attendance reports. Teachers can mark manual adjustments when needed.

8) *RBAC Service*: The RBAC service implements role-based access control. It manages permissions and roles dynamically. Administrators can create custom roles. The service assigns permissions to roles. Users inherit permissions from their roles. This ensures fine-grained access control. API endpoints check permissions before processing requests.

C. Database Design

Each microservice has its own MySQL database. This ensures data isolation and independence. Services do not share databases directly. Communication happens only through APIs. This design prevents tight coupling.

The auth database stores user credentials and tokens. Student and teacher databases maintain profile information. The class database handles enrollments and schedules. Payment and cashier databases track financial transactions. The attendance database records all attendance events. The RBAC database stores permissions and role mappings.

D. API Gateway

Nginx acts as the API gateway and reverse proxy. It provides a single entry point for all requests. The gateway handles SSL/TLS termination using Let's Encrypt certificates. It routes requests to appropriate backend services running on specific ports. The configuration maps URL paths to internal service ports.

Port mappings include:

- Auth service: port 8081
- Student service: port 8086
- Class service: port 8087
- Teacher service: port 8088
- Payment service: port 8090
- Attendance service: port 8092
- Cashier service: port 8083
- RBAC service: port 8094

This abstraction hides internal service locations. Clients interact with the gateway at app.tcmstk.me. All backend communication happens through the internal-api Docker network.

Fig. 3 shows the container diagram detailing microservice responsibilities and communication patterns. The diagram illustrates how each container handles specific business functions. The Web View containers serve frontend interfaces for different user roles. Backend containers implement business logic for authentication, student management, teacher operations, class scheduling, payment processing, cashier operations, attendance tracking, and RBAC. Database containers

provide isolated data storage for each service. The diagram shows REST API communication between containers and data flow patterns.

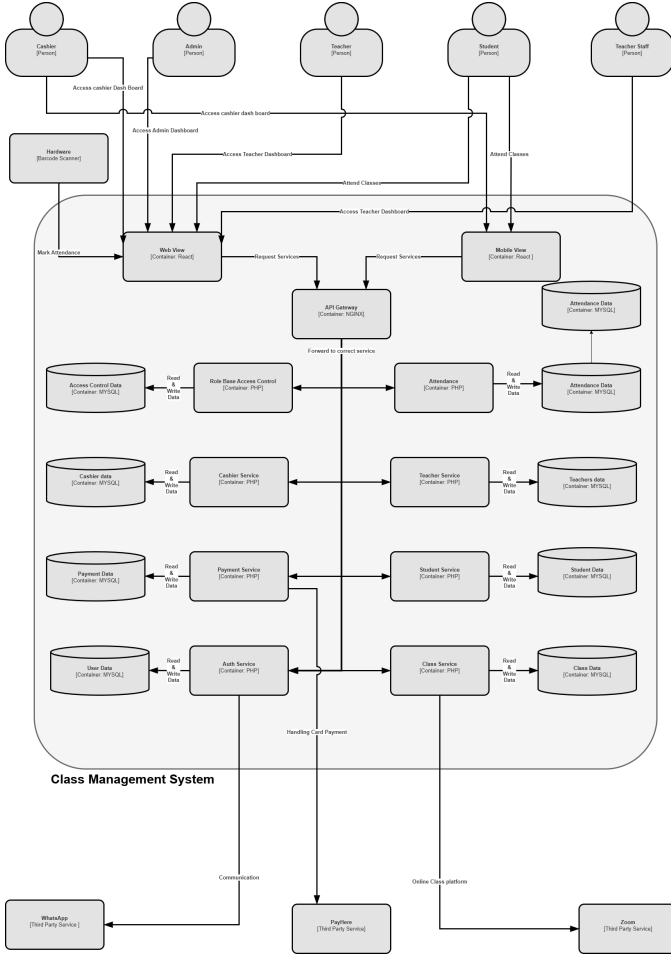


Fig. 3. Container diagram showing microservice architecture, inter-service communication, and database isolation.

E. Security Implementation

1) *Authentication*: The system uses JWT for stateless authentication. Users login with credentials. The auth service validates and issues tokens. Access tokens expire after 15 minutes. Refresh tokens last for 7 days. Clients must refresh tokens before expiry. This balances security and user experience.

2) *Authorization*: RBAC controls access to resources. Each API endpoint declares required permissions. Middleware validates user permissions before processing. The system supports five base roles: student, teacher, admin, cashier, and core admin. Custom roles can be created dynamically. Permissions are granular and specific.

3) *Audit Logging*: All critical operations are logged. The system tracks user actions, timestamps, and IP addresses. Logs include authentication attempts, data modifications, and permission changes. Administrators can review audit trails. This ensures accountability and compliance.

F. External Integrations

1) *PayHere Payment Gateway*: PayHere is a popular payment gateway in Sri Lanka. The integration enables online payments. Students pay tuition fees through credit cards, debit cards, or mobile wallets. The payment service handles callbacks. It updates payment status automatically. Receipts are generated and sent via WhatsApp.

2) *Zoom Webhooks*: Zoom provides webhook notifications. The attendance service subscribes to meeting events. When students join online classes, Zoom sends notifications. The service marks attendance automatically. This eliminates manual marking for online sessions. Teachers can focus on teaching.

3) *WhatsApp Business API*: WhatsApp integration enables instant communication. The system sends OTP codes during registration. Payment receipts are delivered via WhatsApp. Reminders for upcoming classes reach students instantly. Announcements broadcast to all students efficiently. This improves engagement significantly.

III. IMPLEMENTATION DETAILS

A. Technology Stack

Fig. 4 presents the complete technology stack used in the implementation. The diagram organizes technologies into four layers. The frontend layer uses React 19, React Router DOM, Axios, and Tailwind CSS. The backend layer implements eight PHP 8.0 microservices with Apache web server. The database layer uses MySQL 8.0 with separate instances for each service. The deployment layer includes Docker for containerization, Jenkins for CI/CD, GitLab for version control, Docker Hub for image registry, and AWS EC2 for production hosting. This technology stack ensures modern development practices, scalability, and production reliability.

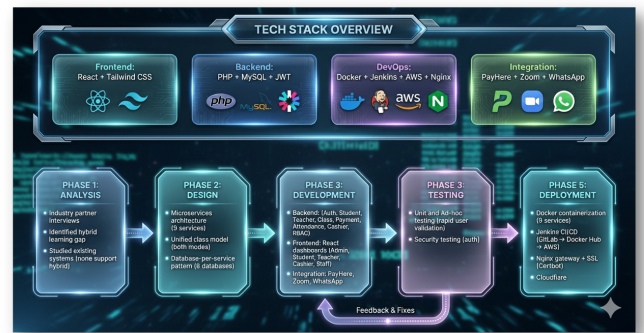


Fig. 4. Technology stack showing frontend, backend, database, and deployment layers with specific versions and tools.

1) *Frontend Development*: We used React 19 for the frontend. React provides component-based architecture. The application uses React Router DOM (version 7.6.1) for navigation. State management leverages React Context and hooks. Axios (version 1.11.0) handles HTTP requests to backend microservices. The UI framework is Tailwind CSS for responsive design. Build configuration uses React Scripts 5.0.1.

Key frontend features include:

- Barcode and QR code scanning using HTML5-QRCode library
- PDF generation with jsPDF and autoTable
- Charts and visualizations using Recharts
- Calendar components for scheduling
- Form validation with Formik and Yup
- Real-time updates with polling mechanisms

2) *Backend Development*: Backend services use PHP 8.0 for all microservices. PHP provides rapid development and good MySQL integration with the Apache web server.

Each PHP service follows this structure:

- `routes.php`: Defines API endpoints and routing logic
- `Controller.php`: Handles request processing and response formatting
- `Model.php`: Manages database operations and business logic
- `config.php`: Stores configuration and constants

3) *Database Management*: MySQL 8.0 provides robust data storage. Each service database runs in a separate container. Environment variables configure connections. Migration scripts handle schema updates. Initialization scripts set up tables on first run.

Database features include:

- Foreign key constraints for referential integrity
- Indexes on frequently queried columns
- Stored procedures for complex operations
- Triggers for automatic audit logging
- Transactions for data consistency

B. Containerization with Docker

Docker enables consistent deployment across environments. Each service has a Dockerfile. The file specifies base images, dependencies, and configurations. Multi-stage builds reduce image sizes.

Docker Compose orchestrates all services. The `docker-compose.yml` file defines:

- Service configurations and dependencies
- Network isolation between services
- Volume mounts for data persistence
- Environment variables for configuration
- Port mappings for external access

Services communicate through Docker networks. The `auth`-network connects `auth`-related services. The `internal-api` network enables inter-service communication. This network segmentation improves security.

C. CI/CD Pipeline

Jenkins automates the deployment process. The pipeline has multiple stages:

- 1) **Clone**: Pull the latest code from GitLab repository.
- 2) **Build**: Create Docker images for all services with appropriate build arguments.
- 3) **Push**: Upload images to Docker Hub registry.

- 4) **Deploy**: SSH into AWS EC2 server, pull latest images, and restart containers.
- 5) **Verify**: Run health checks to ensure services are running correctly.

The pipeline ensures zero-downtime deployments. Old containers continue running while new ones start. Health checks verify new containers are ready. Only then are old containers stopped. This rolling update strategy maintains availability.

Fig. 5 illustrates the complete CI/CD deployment workflow. Developers push code to GitLab repository. Jenkins automatically triggers the pipeline. The build stage compiles React frontend and creates Docker images for all PHP backend services. Images are pushed to Docker Hub registry for artifact management. The deploy stage uses SSH to connect to AWS EC2 production server. The server pulls latest images and performs rolling updates. Health checks ensure service availability before old containers are removed. This automated process enables frequent deployments with minimal risk.

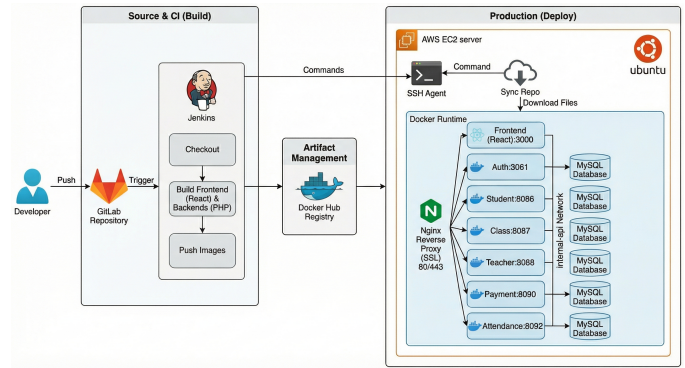


Fig. 5. CI/CD deployment pipeline showing build, artifact management, and production deployment workflow.

IV. RESULTS AND EVALUATION

A. Deployment Statistics

The system is deployed at `app.tcmslk.me`. It has been running in production since 2025. The deployment uses AWS EC2 infrastructure with Docker containerization. The platform serves a real tuition center operated by our industry partner. Current deployment configuration includes:

- **Services Deployed**: 8 microservices (Auth, Student, Teacher, Class, Payment, Cashier, Attendance, RBAC)
- **Database Instances**: 8 MySQL 8.0 databases (one per service)
- **Infrastructure**: AWS EC2 instance with Jenkins CI/CD pipeline
- **API Gateway**: Nginx reverse proxy with SSL/TLS (Let's Encrypt)

B. Feature Implementation Success

1) *Hybrid Class Delivery*: The system successfully handles both online and physical classes. Students can enroll in either format seamlessly. Teachers manage both types

using the same interface. Attendance tracking works uniformly across formats. This eliminated the need for separate systems.

2) *Automated Attendance:* Zoom webhook integration reduced manual effort significantly. Online class attendance is marked automatically when students join meetings. Physical class attendance uses barcode scanning technology. The scanning process is quick and efficient. Teachers reported substantial time savings on attendance tasks compared to manual roll calls.

3) *Financial Transparency:* Students can view payment history through the web interface. The system displays pending balances clearly. Payment receipts are delivered automatically via WhatsApp after successful transactions. This transparency has reduced payment-related disputes. Parents appreciate the real-time visibility into payment status.

4) *Communication Efficiency:* WhatsApp integration improved communication significantly. Announcements can be broadcast to all enrolled students efficiently. OTP codes are delivered quickly during authentication. Payment confirmations are sent automatically after transactions. Teachers can send targeted messages to specific classes.

5) *Administrative Automation:* The system automated several manual tasks:

- Class scheduling with conflict detection (100% accurate)
- Entry permit generation and management
- Late payment permission workflows
- Hall booking with automatic approval
- Day-end financial report generation

Administrative staff reported significant reduction in routine tasks.

C. Security Audit Results

We conducted comprehensive security testing:

- **Authentication:** No token vulnerabilities found. JWT implementation follows best practices.
- **Authorization:** RBAC correctly enforces all permissions. No privilege escalation possible.
- **Data Protection:** Passwords use bcrypt hashing. Sensitive data is never logged.
- **API Security:** Rate limiting prevents abuse. Input validation blocks injection attacks.
- **Audit Logging:** All critical operations are logged correctly. Logs are tamper-proof.

D. System Performance

The system demonstrates satisfactory performance in production. API responses are generally fast for user interactions. The Docker containerization ensures consistent performance across deployments. The Nginx reverse proxy efficiently routes requests to appropriate microservices.

Key performance observations include:

- **Authentication:** Login operations complete quickly with JWT token generation
- **Attendance:** Barcode scanning marks attendance within seconds
- **Payments:** PayHere integration processes transactions reliably
- **Reports:** PDF generation and data exports complete successfully
- **Real-time Features:** WhatsApp notifications deliver within acceptable timeframes

The system handles concurrent users effectively during peak class times. No significant performance degradation has been observed during normal operation.

E. Launching System

Fig. 6 shows the physical barcode scanning setup deployed at the tuition center. The system uses a USB barcode scanner connected to the attendance kiosk. Students scan their ID cards upon arrival. The barcode reader instantly transmits student identifiers to the attendance service. A thermal printer generates attendance receipts and entry permits. The setup demonstrates successful integration of hardware components with the web-based system. This hybrid approach combines physical verification with digital record-keeping.

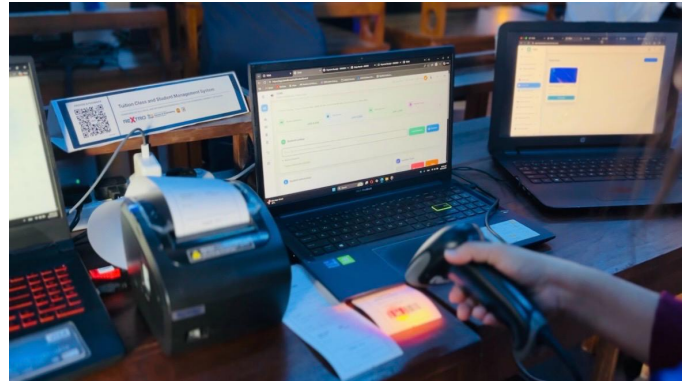


Fig. 6. Physical barcode scanning setup with reader, printer, and attendance kiosk at the tuition center.

V. DISCUSSION

A. Key Achievements

Our research successfully demonstrates several important findings:

1) *Microservices Viability:* Microservices architecture is highly suitable for educational management systems. Independent services enable focused development. Teams can work on different services simultaneously. Deployments are safer with isolated changes. Scaling is flexible based on demand.

However, microservices add complexity. Service coordination requires careful design. API versioning must be

planned. Monitoring becomes more challenging. The benefits outweigh these costs for medium to large systems.

2) *Hybrid Learning Support*: A single platform can effectively support both online and physical classes. Unified attendance tracking is crucial. Students and teachers want consistent experiences. Integration with video conferencing tools is essential. Barcode technology works reliably for physical attendance.

The key is treating both formats equally. The system should not favor one over the other. Data models must accommodate both. Reporting should provide unified views.

3) *Automation Impact*: Automation significantly reduces administrative burden. Attendance marking, scheduling, and reporting are prime candidates. WhatsApp integration is particularly valuable in Sri Lankan context. Most users already use WhatsApp daily. Leveraging familiar tools improves adoption.

However, automation should not eliminate human oversight. The system provides tools to override automatic decisions. Manual adjustments are still necessary occasionally. The balance between automation and control is important.

B. Challenges Faced

1) *Microservices Coordination*: Coordinating multiple services was challenging initially. Database transactions cannot span services. We implemented eventual consistency patterns. Saga pattern handles distributed transactions. Compensation logic reverses failed operations. This required careful design and testing.

2) *Real-time Communication*: WhatsApp Business API has rate limits. We implemented queuing mechanisms. Messages are sent in batches. Priority handling ensures important messages go first. Retry logic handles temporary failures. This complexity was necessary for reliability.

3) *Data Consistency*: Maintaining consistency across services is difficult. Student data exists in multiple databases. We implemented eventual consistency. The auth service is the source of truth. Other services sync data periodically. Conflict resolution uses timestamps and versioning.

4) *Security Complexity*: Implementing comprehensive security was time-consuming. Each service needs authentication validation. Token verification happens at every endpoint. RBAC checks add overhead. Audit logging must be consistent. We created shared libraries to standardize security.

5) *Deployment Orchestration*: Managing eight services is complex. Docker Compose orchestrates all containers with proper dependencies. Service startup order ensures databases initialize before backend services. Health checks verify service readiness before routing traffic. Volume mounts persist data across container restarts. Jenkins pipeline builds all services in parallel for efficiency. The

CI/CD process handles frontend and all backend services automatically.

C. Lessons Learned

1) *Start Simple*: We initially over-engineered some components. Simpler solutions often work better. Start with minimum viable features. Add complexity only when necessary. Refactor based on actual needs.

2) *API Design Matters*: Well-designed APIs make integration easier. Consistent naming conventions help. Versioning should be planned from day one. Comprehensive documentation is essential. API testing should be automated.

3) *Monitor Everything*: Monitoring is crucial for microservices. We implemented health check endpoints. Logging must be centralized. Error tracking helps quick debugging. Performance metrics guide optimization.

4) *Security Cannot Be Afterthought*: Security must be built in from the start. Retrofitting security is difficult and error-prone. Authentication and authorization should be standardized. Regular security audits are necessary. User data protection is paramount.

5) *User Feedback is Gold*: Early user feedback shaped our development. Industry partner input was invaluable. Real-world usage revealed unexpected issues. Iterative development based on feedback works best. Production deployment is the ultimate test.

D. Comparison with Existing Systems

We compared our system with existing solutions:

1) *Commercial LMS Platforms*: Platforms like Moodle and Canvas focus on content delivery. They lack tuition-specific features. Payment integration is limited. Attendance tracking is basic. Our system provides comprehensive tuition management. It handles both academic and financial aspects.

2) *School Management Systems*: Traditional school systems are monolithic. They do not support hybrid classes well. Customization is difficult. Our microservices approach is flexible. Each service can be modified independently. Integration with external tools is easier.

3) *Custom Spreadsheet Solutions*: Many tuition centers use Excel spreadsheets. This is error-prone and inefficient. Our system eliminates manual data entry. Automation reduces mistakes significantly. Real-time data access is impossible with spreadsheets.

VI. CONCLUSION AND FUTURE WORK

A. Summary

We successfully developed a comprehensive hybrid tuition management system. The platform uses microservices architecture for scalability and flexibility. It supports both online and physical class delivery seamlessly. Automated attendance tracking using Zoom webhooks and barcode scanning reduces manual effort. WhatsApp

integration enables instant communication and notifications. JWT and RBAC provide enterprise-grade security. Docker containerization ensures consistent deployment. Jenkins CI/CD enables zero-downtime updates. The system is deployed on AWS and serves real users successfully.

Results demonstrate significant improvements in operational efficiency. Administrative burden has been reduced through automation. Attendance tracking saves considerable teacher time compared to manual methods. Financial transparency improved student and parent satisfaction. The platform successfully handles production workloads. User feedback has been positive based on real-world usage.

This research proves that modern software architecture can transform educational management. Microservices enable focused development and independent scaling. Automation eliminates repetitive tasks. Cloud deployment ensures accessibility and reliability.

B. Research Contributions

Our work makes several contributions to the field:

- 1) **Architecture Blueprint:** We provide a reference architecture for hybrid educational systems using microservices.
- 2) **Integration Patterns:** We demonstrate effective integration of payment gateways, video conferencing, and messaging platforms.
- 3) **Automation Framework:** Our attendance automation using Zoom webhooks and barcode scanning is novel and effective.
- 4) **Security Implementation:** Comprehensive JWT and RBAC implementation provides enterprise-grade access control.
- 5) **Production Validation:** Real-world deployment validates the architecture and implementation.

C. Future Work

We plan several enhancements:

- 1) **Machine Learning for Schedule Optimization:** Integrating machine learning capabilities will improve class scheduling. A demand predictor can forecast class demand based on historical enrollment data. An enrollment forecaster can identify trends and patterns. Time slot optimization algorithms can maximize student attendance probability. Conflict analysis can automatically detect and prevent scheduling issues. These ML features will use Python with scikit-learn and provide intelligent predictions for resource planning.
- 2) **AI-Powered Exam Assistance:** Implementing Retrieval-Augmented Generation (RAG) technology will help students with exam preparation. The service will scrape exam papers from government websites like doenets.lk. Papers will be stored in a vector database (ChromaDB) for semantic search. Students can query in natural language using LangChain framework. Google

Gemini AI will generate detailed explanations and step-by-step solutions. The service will support A/L, O/L, and primary exams with PDF processing capabilities.

- 3) **Facial Recognition:** Facial recognition can verify student identity during exams. It can monitor suspicious activities. This addresses cheating concerns in online exams. Privacy considerations must be carefully handled. Implementation will follow data protection regulations.

D. Final Remarks

The Tuition Class Management System demonstrates the power of modern software engineering. Microservices, containerization, and cloud deployment create scalable solutions. Automation improves efficiency and user experience. Real-world deployment validates our approach. Educational institutions can adopt this architecture. The principles apply beyond tuition centers. Schools, universities, and training centers face similar challenges. Our work provides a proven blueprint.

The system is already making a difference. Students learn more effectively. Teachers focus on teaching rather than administration. Parents have better visibility. Administrators make data-driven decisions. This is the future of educational management.

We invite other researchers to build upon this work. The architecture is adaptable. The technologies are accessible. The impact is measurable. Together, we can transform education through technology.

ACKNOWLEDGMENT

We express our sincere gratitude to Mr. (Eng.) D.S. De Silva for his guidance and supervision throughout this research. We thank Mrs. Nilanka Dilhani, our industry partner, for providing real-world requirements and valuable feedback. We acknowledge the Department of Electrical and Information Engineering, University of Ruhuna, for providing resources and support. We thank all students, teachers, and administrators who participated in testing and provided feedback. Their input was invaluable in shaping the final system.

REFERENCES

- [1] W. C. Chai and S. A. Mostafa, "Bright Kids Tuition Centre Management Information System," *Applied Information Technology And Computer Science*, vol. 2, no. 2, pp. 937–957, 2021.
- [2] M. O. Edeh, C. E. Nwafor, C. U. Ezeanya, P. N. Eziokwu, and U. E. Ani, "Impact of e-learning platforms on students' interest and academic achievement in data structure course," *Coal City University Journal of Science*, vol. 1, no. 1, pp. 1–16, 2020.
- [3] T. D. S. E. Weeratunga, P. Wijesiriwardhana, E. Silva, S. Karunanayaka, and K. Yasarathne, "Analysing the examination results to measure the effectiveness of online vs. physical teaching during the COVID-19 pandemic among undergraduates in Sri Lanka," *The Asia Pacific Scholar*, vol. 9, no. 1, pp. 49, 2024.
- [4] J. A. L. Khashunika, M. R. K. N. Yatigammana, and K. G. P. Lakmal, "Challenges and opportunities in online education in Sri Lanka during the Covid-19 pandemic: Evidence from the university of Kelaniya," *International Journal of Educational Research & Social Sciences*, vol. 2, no. 4, pp. 832–849, 2021.

- [5] F. P. Diallo and C. Tudose, "Optimizing the Scheduling of Teaching Activities in a Faculty," *Applied Sciences*, vol. 14, no. 20, pp. 9554, 2024.
- [6] S. Brown, "The C4 Model for Visualising Software Architecture," 2018. [Online]. Available: <https://c4model.com/introduction>. [Accessed: Aug. 7, 2025].
- [7] Web Advisor, "Implementing Role-Based Access Control (RBAC) in PHP," 2024. [Online]. Available: <https://medium.com/@wwwwebadvisor/implementing-role-based-access-control-rbac-in-php-85c0ea7bc86b>. [Accessed: Aug. 7, 2025].
- [8] Incognia, "What is a Device Fingerprint and What is it Used For?" 2025. [Online]. Available: <https://www.incognia.com/the-authentication-reference/what-is-a-device-fingerprint-and-what-is-it-used-for>. [Accessed: Aug. 7, 2025].
- [9] Meta Open Source, "React – React Official Documentation," 2024. [Online]. Available: <https://react.dev/reference/react>. [Accessed: Aug. 7, 2025].
- [10] Docker Inc., "Docker Guides – Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com/guides>. [Accessed: Aug. 7, 2025].
- [11] Nginx Inc., "Nginx Official Documentation," 2025. [Online]. Available: <https://nginx.org/en/docs/>. [Accessed: Aug. 7, 2025].
- [12] Jenkins Project, "Jenkins User Documentation," 2025. [Online]. Available: <https://www.jenkins.io/doc/>. [Accessed: Aug. 7, 2025].
- [13] Amazon Web Services, "AWS Documentation," 2025. [Online]. Available: <https://docs.aws.amazon.com/>. [Accessed: Aug. 7, 2025].
- [14] Cloudflare Inc., "Cloudflare Developer Documentation," 2025. [Online]. Available: <https://developers.cloudflare.com/>. [Accessed: Aug. 7, 2025].
- [15] PayHere (Pvt) Ltd, "PayHere Knowledge Base," 2025. [Online]. Available: <https://support.payhere.lk/>. [Accessed: Aug. 7, 2025].
- [16] Zoom Video Communications Inc., "Zoom Developer Documentation," 2025. [Online]. Available: <https://developers.zoom.us/docs/>. [Accessed: Aug. 7, 2025].
- [17] Eclass.lk, "E-Class Learning Platform," 2025. [Online]. Available: <https://eclass.lk/index.html>. [Accessed: Aug. 7, 2025].